

Quantum information and quantum computation homework 3

Zhouyue Qian 3220104074 *

(Information and Computational Science Class 2201), Zhejiang University

Due time: December 14, 2024

Abstract

Problem: Prove that any language that can be decided in polynomial time by a Turing machine with an oracle for some language $L \in \Sigma_k$ (or $L \in \Pi_k$) is contained in $\Sigma_{k+1} \cap \Pi_{k+1}$.

Formally, let A be a language and assume there exists a polynomial-time Turing machine M^L that decides A using an oracle for a language $L \in \Sigma_k$ (or $L \in \Pi_k$). Show that:

$$A \in \Sigma_{k+1} \cap \Pi_{k+1}.$$

Definition Recap (Definition 2.2)

Turing Machine with an Oracle (X)

- A Turing machine (TM) uses an oracle as a supplementary tape.
- The oracle can determine the value $X(z)$, where z is the binary representation of an integer j in the range $[0, n-1]$, or $X(z)$ is undefined for other z .

Computation of a Function $f : B^n \rightarrow B^m$

- Input x is provided with $|x| = n$.
- Another number $k \leq m$ is specified, and the computation involves querying the oracle.
- The TM's workspace is limited by s , and $n \leq 2^s$, $m \leq 2^s$.
- The computation halts within $\exp(O(s))$ time or enters a cycle.

Definition of Oracle Turing Machines and Complexity Classes

Definition of Oracle Turing Machines

An oracle Turing machine is a theoretical model that allows a Turing machine to query an external subroutine (oracle) to obtain computational results. The oracle is defined as a language $O \subseteq \{0, 1\}^*$, and its behavior is as follows:

- An oracle Turing machine M^O is a multitape Turing machine with the following additional components:

*Electronic address: 3220104074@edu.zju.cn

- An **oracle tape** for writing the string x to be queried.
- Three special states: $q_{\text{query}}, q_{\text{yes}}, q_{\text{no}}$.
- Operation:
 1. The machine writes a query string x onto the oracle tape.
 2. The machine transitions to the q_{query} state to initiate the query process.
 3. The oracle provides the result:
 - If $x \in O$, the machine transitions to the q_{yes} state.
 - If $x \notin O$, the machine transitions to the q_{no} state.
- The query process is instantaneous, and the computational complexity accounts only for the time required to write x onto the oracle tape.

Definition of Complexity Classes

- P^O :
 - Definition: P^O is the set of all languages computable by a **deterministic Turing machine** (DTM) using oracle O in polynomial time.
 - Mathematical representation:

$$P^O = \{L \subseteq \{0,1\}^* : \exists \text{ DTM } M \text{ such that } M^O \text{ computes } L \text{ in polynomial time}\}$$

- NP^O :
 - Definition: NP^O is the set of all languages computable by a **nondeterministic Turing machine** (NDTM) using oracle O in polynomial time.
 - Mathematical representation:

$$NP^O = \{L \subseteq \{0,1\}^* : \exists \text{ NDTM } M \text{ such that } M^O \text{ computes } L \text{ in polynomial time}\}$$

Role of Oracle Turing Machines in Complexity Classes

Oracle Turing machines play a significant role in the study of complexity classes:

- Oracles provide additional computational power, enabling the simulation of higher levels of complexity classes.
- If the oracle belongs to P , i.e., $O \in P$, then $P^O = P$, as queries to O can be directly simulated by a deterministic Turing machine in polynomial time.
- If the oracle is a more complex language, such as SAT , then NP^{SAT} and P^{SAT} define the computational capabilities involving more complex problems.

Proof Outline

Context and Goal

Assume there is a language A and a polynomial-time Turing machine M^L that decides A using an oracle for some language L . In other words, given an input w , M^L runs in polynomial time and can query the oracle L multiple times (a polynomial number of queries) to decide whether $w \in A$.

We are given that $L \in \Sigma_k$ or $L \in \Pi_k$. Our goal is to show:

$$A \in \Sigma_{k+1} \cap \Pi_{k+1}.$$

Key Definitions

- **Oracle Turing Machine:** An oracle Turing machine M^L can query an oracle for a language L . For any string x , the oracle answers whether $x \in L$ in a single step.
- **Complexity Classes Σ_k and Π_k :** The classes Σ_k and Π_k can be defined in terms of alternating quantifiers over polynomial-time predicates. For instance,

$$L \in \Sigma_k \iff L = \{x \mid \exists y_1 \forall y_2 \exists y_3 \cdots Q y_k : P(x, y_1, \dots, y_k)\}$$

where P is a predicate decidable in polynomial time. Similarly, Π_k classes are defined with an alternating quantifier prefix starting with a universal quantifier. These classes can also be characterized via polynomial-time machines equipped with oracles from lower-level classes.

Proof Idea

1. **Setup:** Suppose M^L is a polynomial-time machine that decides A with access to an L -oracle. Since $L \in \Sigma_k$ (the case $L \in \Pi_k$ is analogous), the oracle queries made by M^L can be replaced by a Σ_k -level decision procedure.

2. **Expanding Oracle Queries:** Each oracle query $x \in L?$ can be decided by a Σ_k -machine (or equivalently represented by a Σ_k -type alternating quantifier formula). Thus, replacing the oracle queries within M^L 's polynomial-time computation amounts to embedding a Σ_k -description of each query's decision procedure.

3. **Complexity Level Upgrade:** If the base machine M (without the oracle) is nondeterministic (corresponding to Σ_0 structure), then adding a Σ_k -level oracle turns the decision process into a Σ_{k+1} -level characterization. Similarly, if M is deterministic (Π_0 -level), the addition of a Σ_k -type oracle also raises the complexity class to Π_{k+1} .

By considering both the Σ_k and Π_k oracle scenarios and using the duality of these classes, we conclude that:

$$A \in \Sigma_{k+1} \cap \Pi_{k+1}.$$

Conclusion

By simulating the oracle queries and integrating the complexity of L (at the Σ_k or Π_k -level) into the polynomial-time computation of M , we effectively increase the complexity level by one. As a result, any language decided in polynomial time by a Turing machine with an oracle for $L \in \Sigma_k$ (or $L \in \Pi_k$) belongs to $\Sigma_{k+1} \cap \Pi_{k+1}$.

$$\boxed{A \in \Sigma_{k+1} \cap \Pi_{k+1}.$$